CACD MINI PROJECT DOCUMENTATION

Security Log Analyzer & Anomaly Detector

Cryptanalysis and Cyber Defence (CACD)

Submitted by:

2420030251	Meduri Rakesh Kumar
2420030650	Kanchi Veera Venkata Ganesh
2420030576	Enumula Hitesh

1. Title

Security Log Analyzer & Anomaly Detector

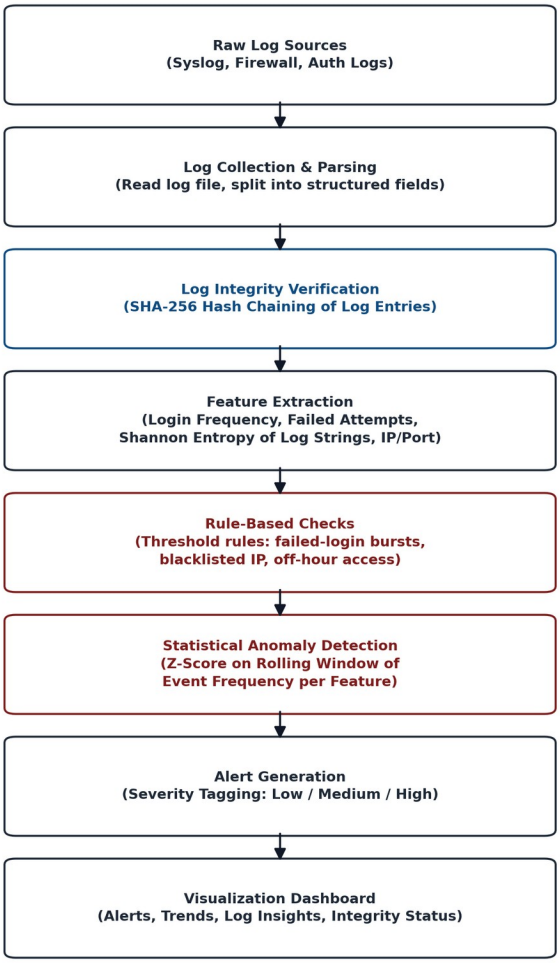
A lightweight system that reads security-relevant logs (authentication, firewall, and system logs), verifies their integrity using SHA-256 hash chaining, extracts simple behavioral features, and flags suspicious activity using rule-based threshold checks combined with basic statistical (Z-score) anomaly detection — no machine learning training required — presented via an interactive dashboard.

2. Objectives

- To design a pipeline that collects and parses security logs (auth logs, firewall logs, system/event logs) into a structured, analyzable format.
- To ensure log integrity using SHA-256 hash chaining, so that any tampering or deletion of past log entries can be cryptographically detected.
- To engineer simple features from raw log data — login frequency, failed-login counts, source IP patterns, access time, and Shannon entropy of log message strings.
- To implement rule-based threshold detection (e.g., failed-login bursts, blacklisted IPs, off-hour access) for known attack patterns.
- To implement statistical (Z-score) anomaly detection on log feature values to catch deviations from normal baseline behavior, without requiring model training or labeled data.
- To generate actionable, severity-tagged alerts (Low / Medium / High) for detected anomalies, including tamper alerts from broken hash chains.
- To build a visualization dashboard that summarizes log trends, flagged anomalies, alerts, and log integrity status in an interpretable form.
- To evaluate detection performance (precision, recall, false-positive rate) on a benchmark intrusion-detection dataset.

3. Flow Chart of the Project

The diagram below illustrates the end-to-end pipeline, from raw log ingestion through preprocessing, feature extraction, anomaly detection, and final visualization.



4. Dataset

The project uses publicly available, widely-cited network intrusion / log datasets so that results can be benchmarked against established literature. Recommended primary and supporting datasets are listed below.

4.1 Primary Dataset — NSL-KDD

An improved version of the classic KDD Cup 1999 dataset with duplicate records removed and a more balanced class distribution. It contains labeled network connection records (normal vs. various attack categories: DoS, Probe, R2L, U2R) and is a standard benchmark for intrusion/anomaly detection coursework and research.

- **NSL-KDD (official — Univ. of New Brunswick, Canadian Institute for Cybersecurity):** https://www.unb.ca/cic/datasets/nsl.html
- **NSL-KDD mirror (Kaggle, CSV format, ready to download):** https://www.kaggle.com/datasets/hassan06/nslkdd

4.2 Supporting / Alternative Dataset — CICIDS2017

A more modern, realistic intrusion detection dataset generated by the Canadian Institute for Cybersecurity, containing benign traffic and common attacks (Brute Force, DoS, DDoS, Infiltration, Botnet, Web Attacks) with full packet payloads and labeled flows — useful if the project is extended to network flow-based anomaly detection.

- **CICIDS2017 (official):** https://www.unb.ca/cic/datasets/ids-2017.html
- **CICIDS2017 mirror (Kaggle, preprocessed CSVs):** https://www.kaggle.com/datasets/cicdataset/cicids2017

4.3 Optional — Real System/Auth Log Simulation

For a lighter-weight, log-file-centric version of the project (closer to a SIEM-style log analyzer rather than a network IDS), the Linux authentication log dataset below can be used or supplemented with synthetically generated auth logs.

- **Loghub — collection of real-world system log datasets (SSH, Apache, Linux, Windows, Hadoop, etc.):** https://github.com/logpai/loghub

Note: dataset hosting pages occasionally reorganize their links. If a link above has moved, search the dataset name directly on the host site (UNB CIC Datasets page, or Kaggle) — the dataset names themselves are stable identifiers.

5. Methodology Overview

This project intentionally avoids training a machine learning model, so it can be implemented quickly and explained easily. Detection is done using two simple, well-established techniques — rule-based threshold checks and Z-score statistical anomaly detection — combined with two cryptography/cryptanalysis techniques that tie the project back to the CACD subject: SHA-256 hash chaining for log integrity, and Shannon entropy as an anomaly feature.

5.1 Log Collection & Parsing

- Load the chosen dataset / log file (NSL-KDD CSV or raw auth/firewall log lines) and parse each entry into structured fields: timestamp, source IP, user, event type, status (success/fail).
- Store parsed entries in a pandas DataFrame for easy filtering and aggregation.

5.2 Log Integrity Verification (SHA-256 Hash Chaining)

- As each log entry is ingested, compute its SHA-256 hash concatenated with the hash of the previous entry: hash(n) = SHA256(entry(n) + hash(n-1)). This links every entry into a tamper-evident chain, the same principle used in blockchain ledgers.
- Store the computed hash chain alongside the parsed logs (e.g., in a separate integrity log or database column).
- To verify integrity at any time, recompute the chain from the stored raw entries and compare it against the saved hash values — if even one character of one past entry has been altered, deleted, or reordered, the recomputed hash will not match from that point onward.

- Any mismatch is raised immediately as a High-severity 'Log Tampering Detected' alert, independent of the anomaly detection logic — this protects the evidentiary value of the logs themselves, not just the events they describe.
- Implementation is a few lines of Python using the built-in hashlib module (hashlib.sha256()), so it adds negligible development effort.

5.3 Feature Extraction

- Login frequency and failed-login count per user/IP within a rolling time window (e.g., per minute).
- Unique source IPs per user, and access outside normal working hours.
- Shannon entropy of log message / payload strings — higher entropy suggests encoded or obfuscated content, a technique borrowed directly from cryptanalysis.
- Formula: $H(X) = -\Sigma\, p(x) \cdot \log2\, p(x)$, computed over the character distribution of each log string.

5.4 Rule-Based Detection

- Apply fixed threshold rules, e.g.: more than 5 failed logins in 60 seconds → flag; source IP on a blacklist → flag; login attempt outside 9 AM–9 PM → flag.
- Rule-based checks require no training data and directly catch known attack patterns (brute force, blacklisted access).

5.5 Statistical Anomaly Detection (Z-Score)

- For each numeric feature (e.g., events per minute, entropy value), compute the rolling mean (μ) and standard deviation (σ) over a baseline window.
- Compute $Z = (x - \mu) / \sigma$ for each new value; flag as anomalous if $|Z| > 3$ (a standard, adjustable threshold).
- This requires no model training — only basic statistics — making it fast to implement and easy to justify in a viva.

5.6 Alerting & Visualization

- Combine rule-based flags, Z-score flags, and hash-chain integrity checks into a single alert feed with a severity tag: High (tampering detected, or both rule + Z-score triggered), Medium (one detection method triggered), Low (borderline Z-score).
- Build a simple dashboard (Streamlit) showing: alerts table, anomaly trend over time, top offending IPs/users, and a log integrity status indicator (Verified / Tampered).

5.7 Evaluation

- Using the labeled NSL-KDD dataset, compute precision, recall, F1-score, and false-positive rate by comparing flagged entries against ground-truth attack labels.
- Separately test the integrity module by deliberately editing a past log entry and confirming the hash chain correctly flags the tampering.

5.8 Tools & Tech Stack (suggested)

- Python: pandas, numpy (for parsing, rolling windows, Z-score, entropy calculation), hashlib (for SHA-256 hash chaining)

- Visualization: Streamlit (simplest option for a quick interactive dashboard)
- No ML framework (scikit-learn/TensorFlow) is required for this basic version — it can be added later as an extension if time permits.